

ФГБОУ ВО «Ивановский государственный
энергетический университет имени В.И. Ленина»
Факультет информатики и вычислительной техники
Кафедра программного обеспечения компьютерных систем

Отчет по лабораторной работе

по дисциплине “Алгоритмы и структуры данных”

Выполнил:

Ст. гр. 3-42 Лашманов В.С

(подпись)

(дата)

Руководитель:

Преподаватель каф. ПОКС Пантелеев Е.Р.

(подпись)

(дата)

Иваново, 2024

Постановка задачи

Задача о максимальном потоке. Построение потокодобавляющего пути методом поиска в ширину. Тестирование алгоритма.

Решение

Задача о максимальном потоке состоит в поиске такой функции потокораспределения f , которая максимизирует суммарный поток из истока (суммарный поток в сток).

Реализация программы

Для решения этой задачи мы воспользуемся алгоритмом Эдмондса–Карпа. Этот алгоритм является улучшенной версией алгоритма Форда–Фалкерсона, где на каждом шаге мы ищем самый короткий путь от истока (s) до стока (t) в остаточной сети. Чтобы найти этот путь, мы применяем поиск в ширину.

Сеть представляет собой ориентированный граф, где каждое ребро имеет пропускную способность. Задача заключается в нахождении максимального потока, который может быть передан от истока ($source$) к стоку ($sink$) с учетом пропускных способностей ребер.

Основные компоненты реализации включают:

Класс `Graph`:

- Хранит граф в виде списка смежности, где каждый узел содержит `unordered_map`, который отображает соседей на пару значений: пропускную способность и текущий поток.
- Метод `addEdge` добавляет ребро в граф, устанавливая пропускную способность и поток. Для обратных ребер пропускная способность устанавливается в 0, а поток — в отрицательное значение.

Метод `bfs`:

- Используется для поиска увеличивающего пути (`augmenting path`) в остаточном графе.
- Возвращает `true`, если путь найден, и обновляет массив `path`, который хранит путь от истока к стоку.

Метод `solution`:

- Основной метод, реализующий алгоритм Форда–Фалкерсона.
- Повторяет поиск увеличивающего пути с помощью `bfs` и обновляет поток вдоль найденного пути, пока такие пути существуют.
- Выводит информацию о найденных путях, обновлении потоков и текущем максимальном потоке.

Метод printFlows:

- Выводит текущие потоки и пропускные способности для всех ребер графа.

Шаги алгоритма:

1. Инициализация:

- Создается остаточный граф, который изначально совпадает с исходным графом, но пропускные способности в остаточном графе равны разнице между пропускной способностью и текущим потоком.

```
vector<unordered_map<int, int>> residualGraph(n);  
for (int u = 0; u < n; ++u) {  
    for (const auto& neighbor : graph[u]) {  
        int v = neighbor.first;  
        int capacity = neighbor.second.first;  
        int flow = neighbor.second.second;  
        residualGraph[u][v] = capacity - flow;  
    }  
}
```

2. Поиск увеличивающего пути:

- Используется алгоритм поиска в ширину (BFS) для нахождения пути от истока к стоку в остаточном графе.
- Если путь найден, то минимальная пропускная способность ребер вдоль этого пути определяет максимальный дополнительный поток, который можно пропустить.

```
while (bfs(residualGraph, source, sink, path)) {  
    int pathFlow = INT_MAX;  
    for (int v = sink; v != source; v = path[v]) {  
        int u = path[v];  
        pathFlow = min(pathFlow, residualGraph[u].at(v));  
    }  
}
```

3. Обновление потоков:

- Поток вдоль найденного пути увеличивается на величину минимальной пропускной способности.
- Обновляются потоки в прямых и обратных ребрах:
- Для прямого ребра поток увеличивается.
- Для обратного ребра поток уменьшается (это позволяет алгоритму "откатить" поток, если это необходимо).

```
for (int v = sink; v != source; v = path[v]) {  
    int u = path[v];  
    graph[u][v].second += pathFlow;  
    graph[v][u].second -= pathFlow;  
  
    residualGraph[u][v] -= pathFlow;  
    residualGraph[v][u] += pathFlow;  
}
```

4. Повторение:

- Шаги 2 и 3 повторяются до тех пор, пока в остаточном графе существует путь от истока к стоку.

```
maxFlow += pathFlow;
```

Отличия от отчета 4.1.

В класс Graph было добавлено поле operationCount типа int, которое служит счетчиком операций. Это поле увеличивается в методе поиска в ширину (bfs) и в методе решения задачи о максимальном потоке (solution).

Для получения состояния графа были добавлены следующие селекторы:

- int get_operation_count() — возвращает текущее значение счетчика операций operationCount.
- int get_edge_count() — возвращает количество ребер в графе.
- int get_node_count() — возвращает количество вершин в графе.

Также был реализован метод `generateRandomGraph(int n, int maxCapacity)`, который генерирует случайную транспортную сеть. Этот метод принимает два параметра:

- `n` — количество вершин в графе.
- `maxCapacity` — максимальное значение пропускной способности ребер.

```
void generateRandomGraph(int n, int maxCapacity) {
```

```
    // Создаем минимальное количество ребер для гарантии существования пути от  
    истока к стоку
```

```
    for (int i = 0; i < n - 1; i++) {  
        addEdge(i, i + 1, rand() % maxCapacity + 1);  
    }
```

```
    // Добавляем дополнительные ребра с вероятностью 0.5
```

```
    for (int u = 0; u < n - 1; ++u) {  
        for (int v = u + 2; v < n; ++v) {  
            if (rand() % 2 == 0) {  
                int capacity = rand() % maxCapacity + 1;  
                addEdge(u, v, capacity);  
            }  
        }  
    }  
}
```

Описание работы метода:

1. Минимальный путь от истока к стоку:

Первый цикл создает ребра вида $(0 \rightarrow 1)$, $(1 \rightarrow 2)$, ..., $(n-2 \rightarrow n-1)$, что гарантирует наличие хотя бы одного пути от истока (вершина 0) к стоку (вершина $n-1$).

2. Дополнительные ребра:

Второй цикл добавляет дополнительные ребра между вершинами u и v с вероятностью 0.5.

Вложенный цикл начинается с $v = u + 2$, чтобы избежать добавления уже существующих ребер и петель (ребер вида $(u \rightarrow u)$).

3. Свойства истока и стока:

Вершина с индексом $n-1$ (сток) всегда имеет нулевую полустепень исхода, так как внешний цикл завершается на вершине $n-2$, и ребро $(n-1 \rightarrow x)$ никогда не создается.

Вершина с индексом 0 (исток) всегда имеет нулевую полустепень захода, так как она никогда не используется как конечная точка ребер.

Добавление дополнительных ребер:

```
for (int u = 0; u < n - 1; ++u) {  
    for (int v = u + 2; v < n; ++v) {  
        if (rand() % 2 == 0) {  
            int capacity = rand() % maxCapacity + 1;  
            addEdge(u, v, capacity);  
        }  
    }  
}
```

Определение транспортной сети:

Транспортная сеть — это ориентированный граф (V, E) со следующими свойствами:

- В графе нет ребер, соединяющих вершину саму с собой.
- Существует единственная вершина с нулевой полустепенью захода (исток S).
- Существует единственная вершина с нулевой полустепенью исхода (сток T).
- На каждом ребре e определены целочисленные неотрицательные функции:
 - $C(e)$ — пропускная способность ребра.
 - $f(e)$ — поток через ребро, причем $0 \leq f(e) \leq C(e)$.

Если $f(e) = C(e)$, ребро считается насыщенным потоком.

- Для всех вершин $V \setminus \{S, T\}$ сумма входящих потоков равна сумме исходящих потоков.

Вывод:

Генерация транспортной сети с помощью метода `generateRandomGraph` учитывает все вышеперечисленные свойства. Таким образом, можно утверждать, что транспортная сеть генерируется корректно, так как алгоритм соответствует всем требованиям определения транспортной сети.

Испытания

1. Нахождение пути: Алгоритм Эдмондса — Карпа использует поиск в ширину (BFS) для нахождения кратчайшего увеличивающего пути в остаточной сети. Поиск в ширину выполняется за время $O(|E|)$, где $|E|$ — количество рёбер в графе.
2. Количество путей: В алгоритме Эдмондса — Карпа количество увеличивающих путей ограничено значением $O(|V| \cdot |E|)$. Это связано с тем, что каждый увеличивающий путь увеличивает поток, и длина кратчайшего пути (по количеству рёбер) может увеличиваться не более чем $O(|V|)$ раз. Для каждого уровня длины пути может быть выполнено не более $O(|E|)$ увеличений.
3. Общая сложность: Поскольку каждый поиск пути занимает $O(|E|)$, а общее количество увеличений составляет $O(|V| \cdot |E|)$, то общая сложность алгоритма Эдмондса — Карпа равна:

$$O(|E|) \cdot O(|V| \cdot |E|) = O(|V| \cdot |E|^2).$$

В функции main будут сгенерированы 10 графов, каждый из которых содержит 20, 40, 60, 80, 100, 120, 140, 160, 180, 200 вершин. Для каждого графа будет применен алгоритм поиска максимального потока, и будут выведены следующие данные: количество вершин, количество ребер, теоретическая оценка сложности и количество операций, выполненных алгоритмом. Результат выполнения программы представлен на рисунке 1.

Количество вершин	Количество ребер	Аналитическая оценка	Количество операций	Передача меток	Пересчёт потока	Обновление потока	Сравнения
20	108	43200	1319	218	31	31	1039
Максимальный поток: 83							
40	412	659200	17200	1396	137	137	15530
Максимальный поток: 646							
60	889	3200400	23573	1792	128	128	21525
Максимальный поток: 221							
80	1591	10182400	126526	5762	387	387	119990
Максимальный поток: 842							
100	2570	25700000	216607	8227	288	288	207804
Максимальный поток: 1108							
120	3598	51811200	400321	14967	622	622	384110
Максимальный поток: 789							
140	5030	98588000	626910	17359	520	520	608511
Максимальный поток: 558							
160	6459	165350400	798751	22526	838	838	774549
Максимальный поток: 2444							
180	8048	260755200	701483	23642	578	578	676685
Максимальный поток: 2101							
200	10030	401200000	1261347	34985	778	778	1224806
Максимальный поток: 1421							

Рисунок 1 – Вывод

На основе полученных данных (рисунок 1) построим график в Excel (рисунок 2).

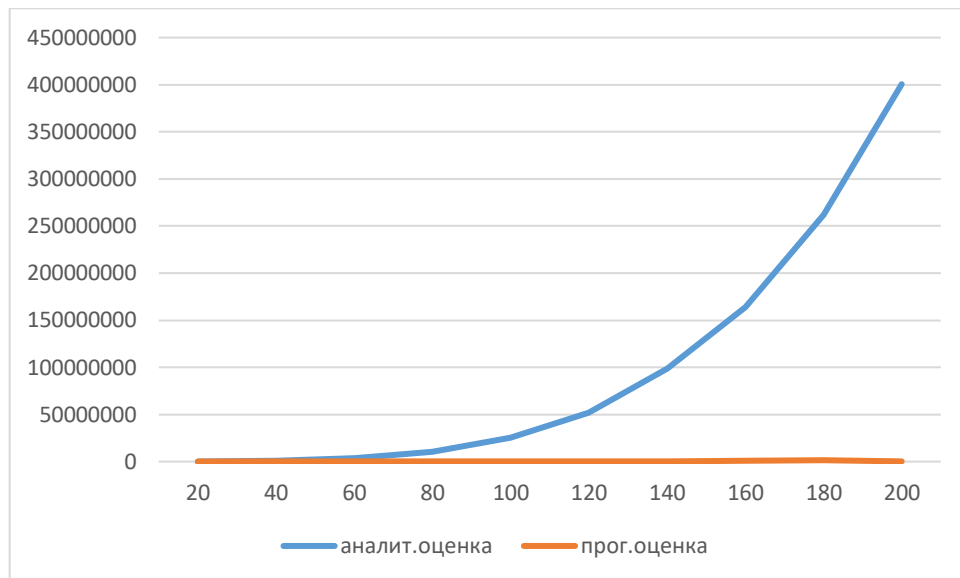


Рисунок 2 – График зависимости теоретической и экспериментальной сложности алгоритма

На графике синим цветом изображена кривая, соответствующая теоретической оценке эффективности алгоритма, а оранжевым цветом – кривая, построенная на основе экспериментальных данных, полученных с использованием счетчика операций.

Аналитическая оценка больше, чем программная оценка, потому что:

1. Аналитическая оценка — это верхняя граница. Она учитывает худший случай работы алгоритма, где количество операций максимально. Например, в вашем коде оценка $\text{edgeCount} * (i * i)$ предполагает, что алгоритм будет работать в самом неблагоприятном сценарии.
2. Реальные данные проще. На практике графы часто имеют структуру, которая позволяет алгоритму завершить работу быстрее, чем в худшем случае. Например, если путь для увеличения потока находится быстро, количество операций уменьшается.
3. Аналитическая оценка не учитывает оптимизации. В реальной реализации могут быть оптимизации или особенности данных, которые снижают количество операций. Например, если поток достигает максимума за меньшее число итераций.

Аналитическая оценка переоценивает сложность, чтобы гарантировать, что алгоритм справится даже в худшем случае, а программная оценка отражает реальное поведение на конкретных данных.

Вывод

Экспериментальные данные показали, что с увеличением числа вершин и рёбер в графе количество операций возрастало, что согласуется с теоретической оценкой. Однако фактическая эффективность алгоритма оказалась выше, чем предполагалось. Это связано с тем, что теоретическая оценка учитывает худший случай, где все рёбра активно используются, а на практике не все рёбра задействованы в полной мере. Таким образом, реальная производительность алгоритма превзошла теоретические ожидания, подтвердив его эффективность для рассмотренных случаев.

Приложение

```
#include <iostream>
#include <vector>
#include <queue>
#include <unordered_map>
#include <limits.h>
#include <locale>
#include <iomanip>
#include <chrono>
#include <cstdlib>
#include <ctime>

using namespace std;

class Graph {
private:
    vector<unordered_map<int, pair<int, int>>> graph;
    size_t labelTransfers = 0; // Счётчик операций передачи меток
    size_t flowOperations = 0; // Счётчик операций пересчёта потока
    size_t flowUpdates = 0; // Счётчик операций обновления потока
    size_t comparisonsOp = 0; // Счётчик операций сравнения

    bool bfs(const vector<unordered_map<int, int>>& residualGraph, int source, int sink, vector<int>& path) {
        size_t n = residualGraph.size();
        vector<bool> visited(n, false);
        queue<int> q;
        q.push(source);
        visited[source] = true;
        path[source] = -1;

        while (!q.empty()) {
            int u = q.front();
            q.pop();

            for (const auto& neighbor : residualGraph[u]) {
                int v = neighbor.first;
                int capacity = neighbor.second;

                comparisonsOp++;
                if (!visited[v] && capacity > 0) {
                    q.push(v);
                    path[v] = u;
                    visited[v] = true;
                    labelTransfers++;
                    if (v == sink) {
                        return true;
                    }
                }
            }
        }
        return false;
    }
};
```

```

    }
    }
}
return false;
}

```

public:

```

Graph(int numVertices) : graph(numVertices) {}

void addEdge(int from, int to, int capacity, int flow = 0) {
    graph[from][to] = { capacity, flow };
    graph[to][from] = { 0, -flow }; // Обратное ребро
}

int solution(int source, int sink) {
    size_t n = graph.size();

    vector<unordered_map<int, int>> residualGraph(n);
    for (size_t u = 0; u < n; ++u) {
        for (const auto& neighbor : graph[u]) {
            int v = neighbor.first;
            int capacity = neighbor.second.first;
            int flow = neighbor.second.second;
            residualGraph[u][v] = capacity - flow;
        }
    }

    vector<int> path(n, -1);
    int maxFlow = 0;

    while (bfs(residualGraph, source, sink, path)) {
        int pathFlow = INT_MAX;
        for (int v = sink; v != source; v = path[v]) {
            int u = path[v];
            pathFlow = min(pathFlow, residualGraph[u].at(v));
            flowOperations++;
        }

        for (int v = sink; v != source; v = path[v]) {
            int u = path[v];
            graph[u][v].second += pathFlow;
            graph[v][u].second -= pathFlow;

            residualGraph[u][v] -= pathFlow;
            residualGraph[v][u] += pathFlow;

            flowUpdates++;
        }

        maxFlow += pathFlow;
    }

    return maxFlow;
}

void generateRandomGraph(int n, int maxCapacity) {
    for (int i = 0; i < n - 1; i++) {
        addEdge(i, i + 1, rand() % maxCapacity + 1);
    }

    for (int u = 0; u < n - 1; ++u) {

```

```
for (int v = u + 2; v < n; ++v) {
    if (rand() % 2 == 0) {
        int capacity = rand() % maxCapacity + 1;
        addEdge(u, v, capacity);
    }
}
}
```

```
int get_edge_count() {
    size_t e = 0;
    for (const auto& node : graph) {
        e += node.size();
    }
    return static_cast<int>(e / 2); // Каждое ребро учтено дважды
}
```

```
int get_node_count() {
    return static_cast<int>(graph.size());
}
```

```
int getLabelTransfers() const { return labelTransfers; }
```

```
int getFlowOperations() const { return flowOperations; }
```

```
int getFlowUpdates() const { return flowUpdates; }
```

```
int getComparisonOps() const { return comparisonsOp; }
```

```
void resetCounters() {
    labelTransfers = 0;
    flowOperations = 0;
    flowUpdates = 0;
    comparisonsOp = 0;
}
};
```

```
int main() {
    setlocale(LC_ALL, "");
    srand(static_cast<unsigned int>(time(0)));

    cout << "-----+-----+-----+-----+-----+-----+-----+-----+-----+-----\n";
    cout << "| Количество вершин | Количество ребер | Аналитическая оценка | Количество операций | Передача\nметок | Пересчёт потока | Обновление потока | Сравнения |\n";
    cout << "-----+-----+-----+-----+-----+-----+-----+-----+-----+-----\n";

    for (int i = 20; i <= 200; i += 20) {
        Graph g(i);
        g.generateRandomGraph(i, rand() % 100 + 1);

        g.resetCounters();
        int maxFlow = g.solution(0, i - 1);

        long long edgeCount = g.get_edge_count();
        long long analyticalEstimate = edgeCount * (i * i);

        cout << " | " << setw(16) << i
             << " " << setw(16) << edgeCount
             << " " << setw(19) << analyticalEstimate
             << " " << setw(18) << g.getLabelTransfers() + g.getFlowOperations() + g.getFlowUpdates() + g.getComparisonOps()
```

```

        << " " << setw(15) << g.getLabelTransfers()
        << " " << setw(15) << g.getFlowOperations()
        << " " << setw(15) << g.getFlowUpdates()
        << " " << setw(15) << g.getComparisonOps() << " |" << endl;

    cout << "Максимальный поток: " << maxFlow << "\n" << endl;
}

cout << "+-----+-----+-----+-----+-----+-----+-----+-----" <<
endl;

return 0;
}

```